Stochastic declustering, from the problem to the algorithm

Companion to `etas8py`, a port of Zhuang's Fortran code

5 November 2026

1 The problem

Earthquakes do not arrive independently. A large earthquake is followed by aftershocks, those aftershocks have aftershocks of their own, and a catalogue ends up as a mix of two populations:

- **Background events**, driven by slow tectonic loading. Their rate is roughly constant in time but varies a lot from place to place.
- **Triggered events**, set off by an earlier earthquake. They bunch tightly in time and space around their parent.

Declustering means separating the two. It matters because the clusters hide the long-term signal: to map where stress steadily accumulates, or to ask whether the seismicity rate really changed, you need to look at the background without the aftershock bursts on top of it.

The catalogue does not tell you which event is which. Each row is only a time, a place and a magnitude. Most older methods draw a space-time window around each large event and delete everything inside it; others link events that are close in space and time. Both force a yes/no answer for every event and depend on hand-picked window sizes or link distances.

Stochastic declustering replaces the yes/no with a probability. A statistical model of how earthquakes trigger each other is fitted to the catalogue, and it then gives each event a probability φ_i of being background.

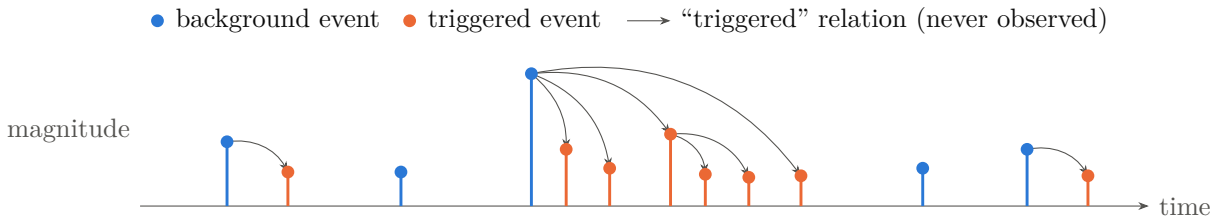


Figure 1: The hidden structure. Real data shows only the stems, with no colours and no arrows. Note the chains: an aftershock can have aftershocks.

The chicken-and-egg difficulty. To label the events you need the background rate and the triggering rules. To estimate those you need the labels. The algorithm breaks the loop by guessing one side, computing the other, and repeating until nothing changes. Sections 2–4 define the pieces and Section 5 shows the loop.

2 Language: rates and likelihood

A **conditional intensity** $\lambda(t, x, y)$ is the expected number of events per day per square degree at that time and place, given everything that happened before. Integrating λ over a region and time

window gives the expected number of events there.

The fit quality of a candidate model on the target events is its **log-likelihood**:

$$\log L \;=\; \sum_{i \,\in\, \text{targets}} \log \lambda(t_i, x_i, y_i) \;-\; \int_{\text{window}} \iint_{S} \lambda(t, x, y)\, \mathrm{d}x\, \mathrm{d}y\, \mathrm{d}t. \tag{1}$$

The first term rewards a high rate where earthquakes did happen. The second term penalises the total number of events the model predicted, so the rate cannot simply be raised everywhere. The second term is what makes this code expensive (Section 6).

3 The model: ETAS

Epidemic-Type Aftershock Sequence treats earthquakes like an epidemic: every event, large or small, can "infect" the future. With $m_j = M_j - M_0$ the magnitude above the catalogue cutoff, the intensity is a background term plus one term per earlier event:

$$\lambda(t, x, y) = \underbrace{\mu\, u(x, y)}_{\text{background}} + \sum_{t_j < t} \kappa_j(t, x, y), \tag{2}$$

$$\kappa_j(t, x, y) = \underbrace{A e^{\alpha m_j}}_{\substack{\text{how many}\\ \text{children}}} \cdot \underbrace{\frac{p-1}{c}\Big(1 + \frac{t - t_j}{c}\Big)^{-p}}_{\text{when: Omori–Utsu decay}} \cdot \underbrace{\frac{q-1}{\pi D e^{\gamma m_j}}\Big(1 + \frac{r_j^2}{D e^{\gamma m_j}}\Big)^{-q}}_{\text{where: spreads with magnitude}} \tag{3}$$

where r_j is the distance to event j. The "when" and "where" factors each integrate to 1, so $A e^{\alpha m_j}$ is the expected total number of children of event j, over all time and space.

Parameter	Meaning	Fitted value (JMA example)
μ	scales the background term	0.551
A	children of an event at the cutoff magnitude	0.166
α	how fast productivity grows with magnitude	1.658
c	short delay before the decay starts	0.0296 days (43 min)
p	how fast the rate decays in time	1.153
D	spatial scale at the cutoff magnitude	0.00183 deg^2 = 22.7 km^2
q	how fast the rate decays with distance	1.951
γ	how fast the spatial scale grows with magnitude	1.067

Table 1: The eight ETAS parameters, $\theta = (\mu, A, c, \alpha, p, D, q, \gamma)$, from `results/cfg_default_flat`.

Those values mean that magnitude dominates everything, as this table of $A e^{\alpha m}$ and $\sqrt{D e^{\gamma m}}$ shows:

Magnitude	4.5	5.5	6.5	7.5	8.2
Expected direct children	0.17	0.87	4.57	24.0	76.5
Spatial scale (km)	4.8	8.1	13.8	23.6	34.3

Table 2: Productivity and spatial scale implied by the fitted parameters.

The background $u(x, y)$. No formula is assumed for the background map. It is built from the data by placing a small Gaussian bump on every event and weighting it by the probability that the event is background:

$$u(x, y) = \frac{1}{T_{\text{window}}} \sum_j \varphi_j\, g_{h_j}(r_j), \qquad g_h(r) = \frac{1}{2\pi h^2}\, e^{-r^2/2h^2}. \tag{4}$$

T_{window} is the length of the target window (13376 days for JMA). This is a **variable-bandwidth kernel density estimate**. The width h_j is the distance from event j to its 5th nearest neighbour, floored at 0.05° (5.6 km). Crowded areas get sharp bumps and empty areas get wide ones. On JMA the median is 7.4 km and the 99th percentile 60 km, and 3542 of the 10071 events sit on the floor. The sum includes the event itself, as in the original.

Equation (4) is where the circularity sits: u needs φ, and φ (next section) needs u.

4 From the model to probabilities

Equation (2) says the rate at event i is a sum of separate causes. If causes add up as rates, the probability that a given cause produced the event is its share of the total:

$$\underbrace{\varphi_i = \frac{\mu\, u(x_i, y_i)}{\lambda(t_i, x_i, y_i)}}_{\text{event } i \text{ is background}} \qquad \underbrace{\rho_{ij} = \frac{\kappa_j(t_i, x_i, y_i)}{\lambda(t_i, x_i, y_i)}}_{\text{event } i \text{ was triggered by } j} \qquad \varphi_i + \sum_{j<i} \rho_{ij} = 1. \tag{5}$$

Three real events from the JMA run. Rates are per day per deg^2.

Event	μu_i	λ_i	φ_i	Largest triggering share ρ_{ij}
#4461, M5.1	0.00185	0.1067	0.017	0.72: a M7.4, 2 days earlier, 64 km away
#7048, M4.6	0.00075	0.00159	0.470	0.51: a M5.0, 0.32 days earlier, 71 km away
#5950, M5.0	0.03091	0.03218	0.960	0.007: a M4.7, 2014 days earlier, 3.5 km away

Table 3: Each row asks what share of the rate at that event comes from the background. The second event is genuinely ambiguous, which a window method cannot express.

Why "stochastic". To get an actual declustered catalogue, draw a uniform random number U_i for each event and keep it as background if $U_i < \varphi_i$. Otherwise it is triggered, and you can pick its parent j with probability $\rho_{ij}/(1-\varphi_i)$. These are Algorithms 2 and 3 of the paper. Each draw gives one equally plausible catalogue, and repeating the draw shows how uncertain any conclusion is. The paper stresses that the essential output is the probabilities, not any one catalogue. This code computes φ_i and stops there: the random draw is not implemented, and the Python port does not write the ρ_{ij} matrix (the original's `pmatr.dat`).

5 The algorithm

The loop has two levels. The inner one solves the circularity between u and φ with the parameters frozen. The outer one re-fits the parameters with u frozen, then goes round again. In spirit this is like expectation–maximisation (assign events to causes, re-fit, repeat), but it is not formally EM. Figure 2 shows the whole thing.

Walking through the boxes

Step 1, prepare. Targets are the events whose fit we score: inside the polygon and inside the target window (1953–1990 for JMA). The other events are not thrown away. Earlier events and those outside the polygon can still trigger targets, so they stay in the sums in Equation (2). JMA keeps 10071 events, of which 4656 are targets. The projection is where `flat` (the original) or `laea` (equal-area) is applied.

Step 2, bandwidths. These depend only on event locations, so they never change.

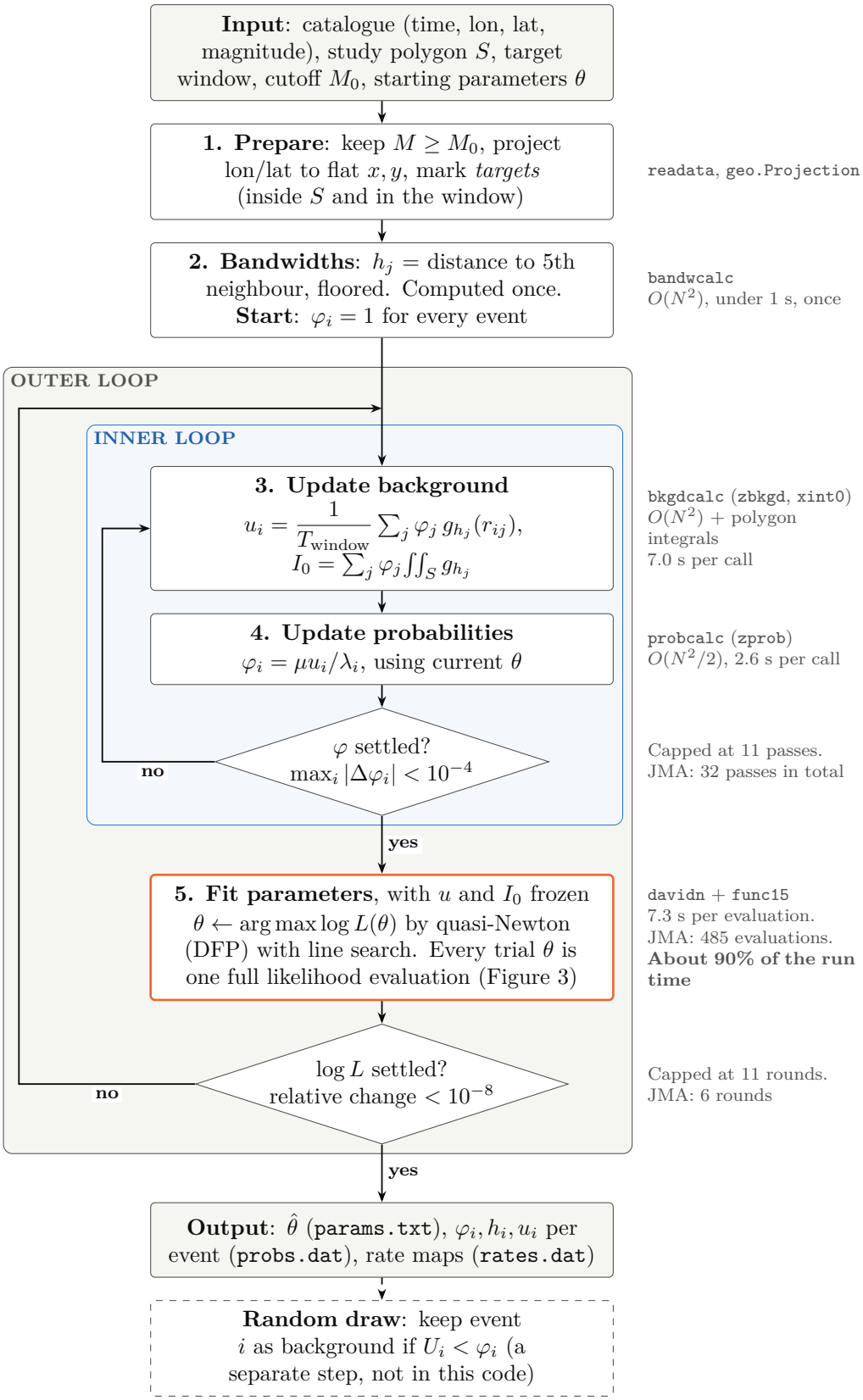


Figure 2: The whole algorithm. Timings are single-thread per-call measurements on the JMA catalogue ($N = 10071$), from the clean rerun in `architecture.md`. The original Fortran has no convergence tests: it runs a fixed schedule of 11 outer rounds with 11, 10, …, 1 inner passes.

Step 3, background. Applies Equation (4) at every event. On the first pass all $\varphi_j = 1$, so u is a map of all seismicity, clusters included. It also computes I_0, the background's weight inside the polygon, which Step 5 reuses.

Step 4, probabilities. Applies Equation (5). Events near big shocks get low φ_i. Back in Step 3 they then contribute less to u, so the map loses its aftershock peaks, and the next φ_i change a little less. On JMA the largest change falls from 1.0 on pass 1 to 4.7×10^{-4} on pass 11.

Step 5, fit. With u frozen, Equation (1) is an ordinary function of eight numbers, and an optimiser climbs it. The optimiser works on $\sqrt{\theta}$, which keeps every parameter positive without constraints. New parameters change λ, hence φ, hence u, so the outer loop goes back to Step 3.

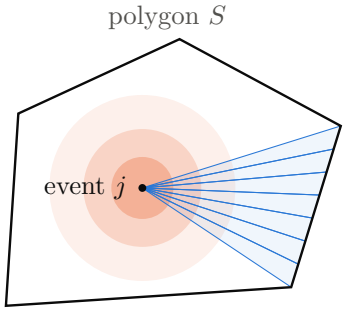
Stopping. The paper stops when the largest change in u falls below a small number. The Fortran has no test and runs a fixed schedule. The tests shown here are this project's choice, and can be turned off to reproduce the Fortran. Both agree closely on JMA: parameters within 1.4×10^{-6} (relative) and φ within 8.4×10^{-6}. The φ_i in `probs.dat` come from the last pass, just before the final fit, as in the original. At convergence that gap is negligible: recomputing the three events in Table 3 with the final θ gives the same values.

6 Why it is slow: the polygon integrals

The penalty term in Equation (1) has to be recomputed for each trial θ:

$$\underbrace{\mu I_0}_{\text{stored from Step 3}} + \sum_j A e^{\alpha m_j} \underbrace{G_j}_{\substack{\text{time integral,} \\ \text{closed form}}} \underbrace{S_j}_{\substack{\text{space integral of (3)'s} \\ \text{“where” factor over } S}} \tag{6}$$

S_j asks: what fraction of event j's aftershocks land inside the study polygon? There is no closed form for that, because the kernel is circular and the polygon is not.



The trick. By Green's theorem the area integral becomes a sum over the boundary. Each edge is cut into 1000 pieces, and each piece forms a thin triangle with the event. A radial kernel has a closed-form antiderivative $F(r)$, so each triangle needs only a one-dimensional Simpson rule in the angle. Triangle areas are signed, so an event outside the polygon also works: parts of the fan cancel. The figure shows the fan for one edge, with 7 pieces rather than 1000.

The cost. 27000 radial evaluations per integral in the original, four integrals per event, 10071 events: about 1.1×10^9 per evaluation, hundreds of times over.

This is why the project's first speedup targeted this step, sharing work between the four integrals and between neighbouring pieces. Full-run time at 10 cores went from 3965 s (Fortran MPI) to 940 s (Python), with results matching to about 10^{-6}. The $O(N^2)$ pair sums are next. Figure 3 shows where this step sits in one likelihood evaluation.

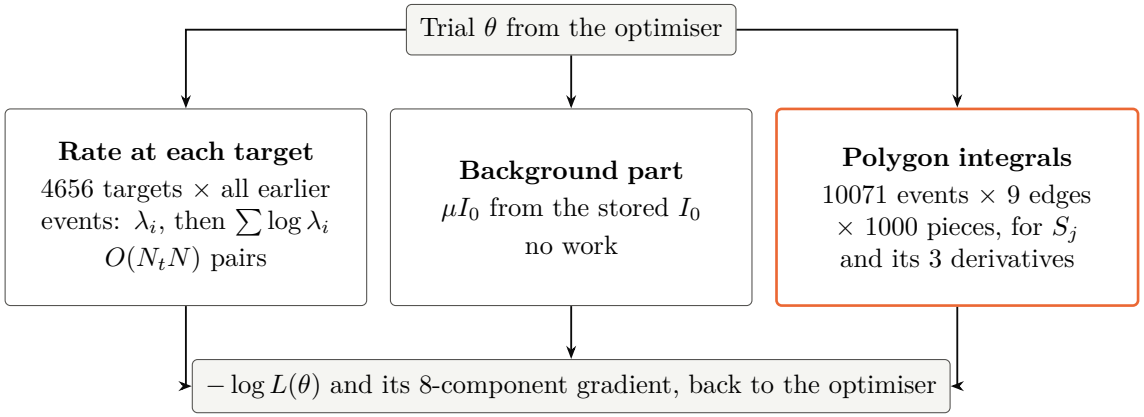


Figure 3: One likelihood evaluation (`func15`). Polygon integrals took 31.0 of 33.5 s in the 1:1 port and 6.6 of 8.7 s after this project's fused version.

7 What comes out on JMA

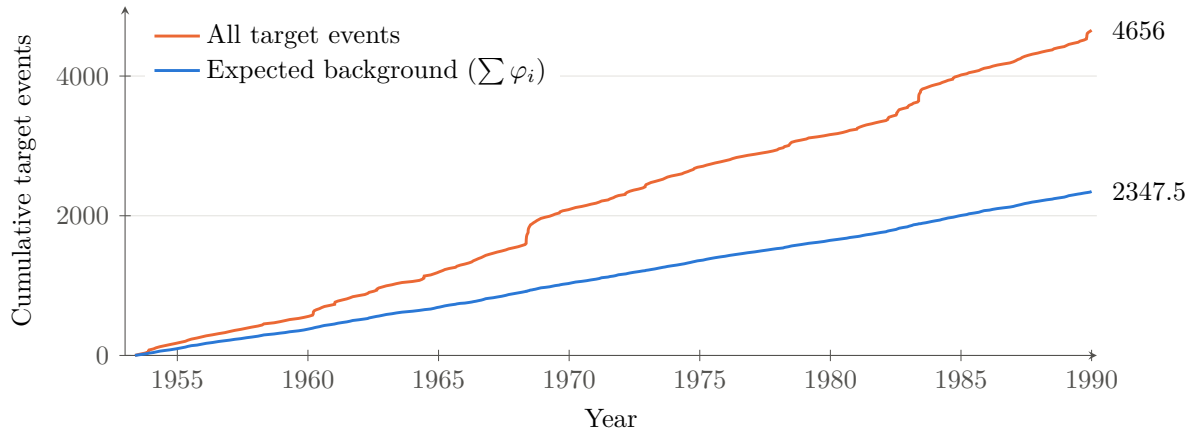
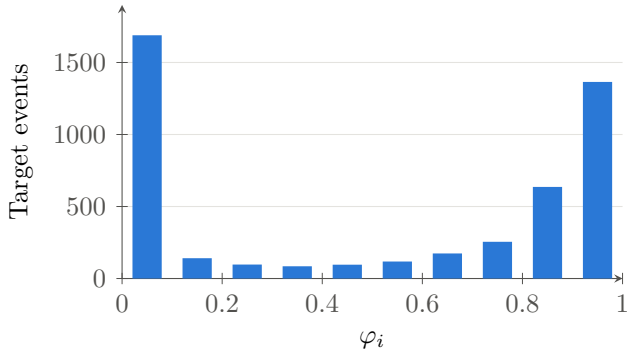


Figure 4: Declustering removes the jumps. The largest step is the M7.9 of 1968: 277 target events in 60 days, but only 19.7 expected background events. The M7.7 of 1983 gives 183 events and 11.3. Source: `results/cfg_default_flat/probs.dat`, `scripts/explainer_data.py`.



Most events are clear-cut. Of 4656 targets, 1689 have $\varphi_i < 0.1$ and 1365 have $\varphi_i > 0.9$. The other 1602 are ambiguous, and a yes/no window method would hide that.

About half is background. Expected background is $\sum \varphi_i = 2347.5$ of 4656, or 50.4%.

A built-in check. Setting $\partial \log L / \partial \mu = 0$ gives $\sum_{\text{targets}} \varphi_i = \mu I_0$ at the optimum. Here the two sides are 2347.5239 and 2347.5237.

Why $\mu \approx 0.55$, not 1. Equation (4) sums over all events since 1926 but divides by the target window only (13376 days). Taken at face value that overstates the daily background rate by about 23376/13376, so μ should come out near $13376/23376 = 0.57$, and it does. The paper supports this: where its sum and its T cover the same period (New Zealand), the same factor converges to 0.9997. It is still my inference, as the paper does not discuss this case. The choice is harmless,

because only the product μu enters the likelihood and φ_i, so μ absorbs any constant rescaling of u.

8 How this code differs from the paper

The model and algorithm in this document follow the code. They are checked here against Zhuang, Ogata & Vere-Jones (2002) and its follow-up (2004). The probabilities φ_i, ρ_{ij}, the weighted kernel estimate and the idea of alternating between u and θ all match the 2002 paper. These details do not:

	2002 paper	This code
Spatial kernel	Gaussian, sharing α with productivity (6 parameters)	Power law with q and its own γ (8 parameters). The 2004 paper has the power law, but still with shared α
Loop	One loop: fit θ, then ρ, then u	Two levels: u and φ until settled, then fit θ
Start	$u = 1$, so the first step is a fit	$\varphi = 1$, so the first step is u
Stopping	$\max \lvert\Delta u\rvert < \varepsilon$	Fortran: fixed schedule. This port: $\Delta\varphi$ and $\Delta \log L$ tests
Bandwidth	$n_p = 20$, floor 0.02°	$n_p = 5$, floor 0.05°, the values suggested in the 2004 paper
T in u	Length of the observation period	Target window only, with the sum over all events
Output	Declustered catalogues and family trees (Algorithms 2, 3)	φ_i only

Table 4: Differences found by reading both papers. I did not trace which later paper introduced the separate γ. The Fortran itself is the source for the code's form.

9 Symbols and where they live in the code

Symbol	Meaning	Code name
μ	scale of the background term; the paper's ν	`b(1)`, `mu`
ρ_{ij}	event i triggered by j; the paper writes $\rho_{j,i}$	not computed
φ_i	probability event i is background	`zprob`, `background_prob`
u_i	background density at event i, per day	`zbkgd`, `background_rate`
h_j	Gaussian bandwidth of event j	`zbandw`, `bandwidth`
I_0	$\sum_j \varphi_j \iint_S g_{h_j}$	`xint0`
λ_i	total intensity at event i	`xlamb`
$G_j S_j$	time and space integrals of event j's offspring	`xint`, `xint_fast`
S_j	one polygon integral	`polyint`, `frint`
$-\log L$	objective and gradient	`func15`
targets	events scored in $\sum \log \lambda_i$	`ind == 1`

References. Zhuang, J., Ogata, Y. & Vere-Jones, D. (2002). Stochastic declustering of space-time earthquake occurrences. *Journal of the American Statistical Association*, 97(458), 369–380.

Zhuang, J., Ogata, Y. & Vere-Jones, D. (2004). Analyzing earthquake clustering features by using stochastic reconstruction. *Journal of Geophysical Research*, 109, B05301.